

Лабораторная работа №9 по JavaScript

Объект canvas, рисование в javascript

Объект canvas

- Для рисования в javascript прежде всего необходимо добавить объект canvas, который имитирует «холст» художника. Данный объект поддерживается всеми браузерами современных версий, но некоторые устаревшие версии не поддерживают его. Значением атрибута id также уместно назначить «canvas»:

```
<canvas id="canvas" width="150" height="150">
```

```
</canvas>
```

- Размеры canvas по умолчанию: 300 px × 150 px (width × height), но можно устанавливать произвольные размеры.
- Для canvas желательно установить какой-либо стиль (css), добавив, например, рамку, или задав задний фон.
- Кроме того, объект может служить контейнером других объектов (например, изображений) и текста:

```
<canvas id="canvas" width="150" height="150">
```

```
current stock price: $3.15 + 0.15
```

```
</canvas>
```

- Главным методом объекта canvas является метод `getContext()`, который используется для получения основных функций рисования. У метода один параметр, указывающий на поддержку типа графики (2d или 3d).

- Добавим javascript-код с обращением к объекту canvas и установкой поддержки 2d-графики:

```
var canvas = document.getElementById('canvas');  
var ctx = canvas.getContext('2d');
```

Проверка поддержки canvas

- Далее добавим javascript-код с обращением к объекту canvas и проверкой поддержки его браузером:

```
var canvas = document.getElementById('canvas');  
if(canvas.getContext){  
    var ctx = canvas.getContext('2d');  
    // работа с канвасом  
} else{  
    // код, не поддерживающий канвас  
}
```

- **Рассмотрим код полностью:**

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <title>Canvas</title>
```

```
    <script type="text/javascript">
```

```
      function draw() {
```

```
        var canvas = document.getElementById('canvas');
```

```
        if (canvas.getContext) {
```

```
          var ctx = canvas.getContext('2d');
```

```
        }
```

```
      }
```

```
    </script>
```

```
    <style type="text/css">
```

```
      canvas { border: 1px solid black; }
```

```
    </style>
```

```
  </head>
```

```
  <body onload="draw();">
```

```
    <canvas id="canvas" width="150" height="150"></canvas>
```

```
  </body>
```

```
</html>
```

Пример: Чтобы проиллюстрировать работу канваса нарисуем два полупрозрачного цвета (красного и синего) квадрата, перекрывающих друг друга.

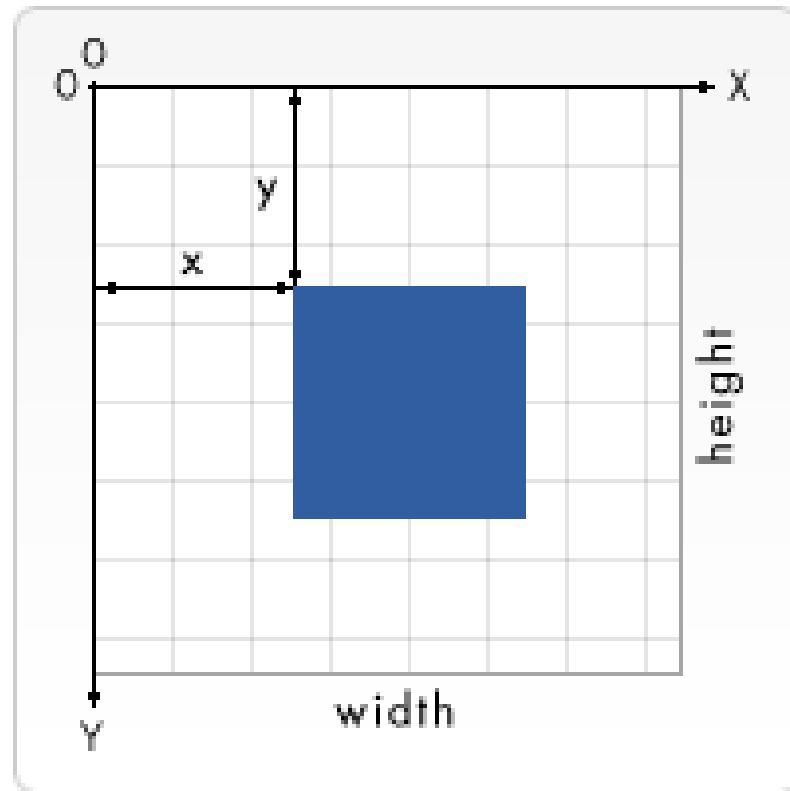
...

```
function draw() {  
    var canvas = document.getElementById('canvas');  
    if (canvas.getContext) {  
        var ctx = canvas.getContext('2d');  
  
        ctx.fillStyle = 'rgb(200, 0, 0)';  
        ctx.fillRect(10, 10, 50, 50);  
  
        ctx.fillStyle = 'rgba(0, 0, 200, 0.5)';  
        ctx.fillRect(30, 30, 50, 50);  
    }  
}
```

...

Рисование фигур (примитивов)

- Отсчет координатной плоскости находится в левом верхнем углу объекта canvas:



- **Canvas поддерживает рисование только одной фигуры — прямоугольника. Остальные фигуры могут получиться из прямых и точек.**

- **Рисование прямоугольника:**

Залитый цветом прямоугольник:

`fillRect(x, y, width, height)`

Прямоугольный контур:

`strokeRect(x, y, width, height)`

Очистка прямоугольной области (прозрачный прямоугольник):

`clearRect(x, y, width, height)`

Выбор цвета в canvas

Для выбора цвета заливки используется метод:

В системе RGB без указания уровня прозрачности:

```
fillStyle = 'rgb(0-255, 0-255, 0-255)';
```

В системе RGB с указанием уровня прозрачности:

```
fillStyle = 'rgba(0-255, 0-255, 0-255, 0.1-0.9)';
```

В качестве заливки по умолчанию стоит черный цвет.

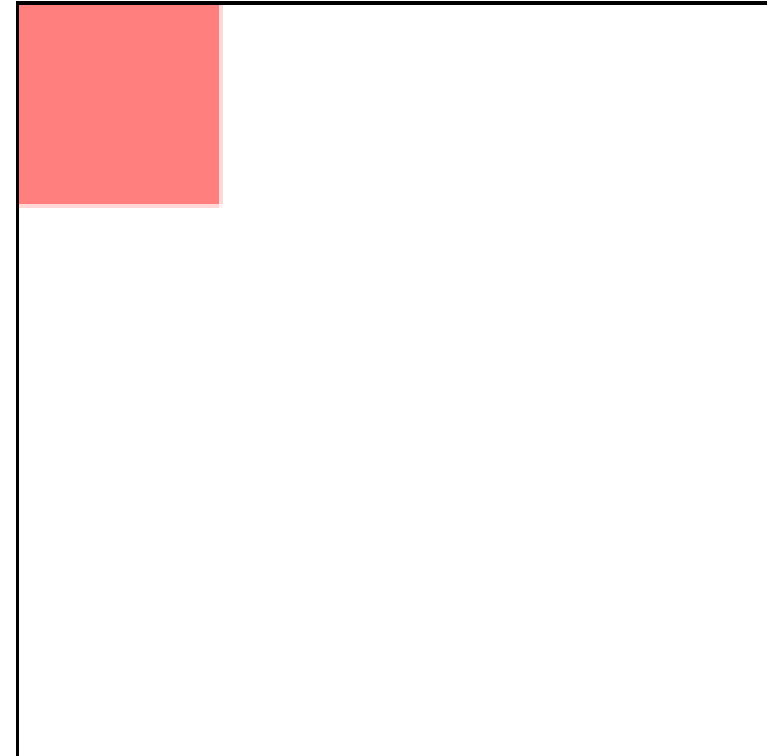
Пример: Нарисовать красный полупрозрачный квадрат 40 x 40, начиная с нулевой координаты.

Решение:

...

```
function draw() {  
    var canvas = document.getElementById('canvas');  
    if (canvas.getContext) {  
        var ctx = canvas.getContext('2d');  
  
        ctx.fillStyle = 'rgba(255, 0, 0, 0.5)';  
        ctx.fillRect(0, 0, 40, 40);  
    }  
}
```

...



Задание js 9.1. Нарисовать горизонтальный ряд квадратов со сторонами 10 на расстоянии 15 от верхнего края канваса и с такими горизонтальными координатами 10, 30, 50, 70, ... , 130.

Рисование путей в canvas

Путь представляет собой набор точек, соединенных прямыми линиями или кривыми. Линии могут быть разной ширины, кривизны и цвета

Рассмотрим этапы построения путей и функции рисования путей в javascript canvas:

1. Создание пути. Когда путь создан остальные команды уже работают над созданным путем:

`beginPath()`

`void ctx.beginPath();`

2. Выбор метода построения пути (кривая, прямая, кривая безье) или переход в новую точку:

- переход в новую точку с указанными координатами:

`CanvasRenderingContext2D.moveTo()`

`void ctx.moveTo(x, y);`

- соединение прямой линией последней использованной координатой с точкой x, y:

`CanvasRenderingContext2D.lineTo()`

`void ctx.lineTo(x, y);`

- **добавление кубической кривой безье** (требуется три точки: первые две (cp1x, cp1y, cp2x, cp2y) — контрольные, последняя — конец линии; начальной точкой является последняя указанная точка, или точка, к которой перешли командой

```
moveTo())
```

```
CanvasRenderingContext2D.bezierCurveTo()
```

```
void ctx.bezierCurveTo(cp1x, cp1y, cp2x, cp2y, x, y);
```

- **добавление квадратичной кривой безье** (cpx — координата x контрольной точки, cpy — координата y контрольной точки):

```
CanvasRenderingContext2D.quadraticCurveTo()
```

```
void ctx.quadraticCurveTo(cpx, cpy, x, y);
```

- **добавление дуги с центром в точке x и y и радиусом r с позиции startAngle до endAngle** (по умолчанию движение дуги по часовой стрелке)

```
CanvasRenderingContext2D.arc()
```

```
void ctx.arc(x, y, radius, startAngle, endAngle [, anticlockwise]);
```

- добавление дуги, соединенной с предыдущей точкой прямой линией:

CanvasRenderingContext2D.arcTo()

void ctx.arcTo(x1, y1, x2, y2, radius);

- добавление эллипса к пути с центром в точке x и y и радиусом radiusX и radiusY с началом в startAngle и окончанием в endAngle:

CanvasRenderingContext2D.ellipse()

void ctx.ellipse(x, y, radiusX, radiusY, rotation, startAngle, endAngle, anticlockwise);

- создание прямоугольного пути:

CanvasRenderingContext2D.rect()

void ctx.rect(x, y, width, height);

3. Конец пути — соединением прямой линией предпоследней точки с начальной точкой пути

closePath()

void ctx.closePath();

4. Рисование пути:

`stroke()`

`void ctx.stroke();`

`void ctx.stroke(path);`

5. Закраска внутренней области пути:

`fill()`

`void ctx.fill();`

`void ctx.fill(path);`

Пример: Нарисовать треугольник красного цвета с гранями разного цвета

Решение:

```
var canvas = document.getElementById('canvas');
```

```
var ctx = canvas.getContext('2d');
```

```
ctx.fillStyle = 'rgb(255, 0, 0)';
```

```
ctx.beginPath(); // начало рисования
```

```
ctx.moveTo(20, 20); // движение к точке 20,20
```

```
ctx.strokeStyle = 'blue';
```

```
ctx.lineTo(200, 20); // из точки 20, 20 рисование линии к точке 200,20
```

```
ctx.strokeStyle = 'green';
```

```
ctx.lineTo(120, 120); // из точки 200, 20 рисование линии к точке 120,120
```

```
ctx.closePath(); // рисование последней линии треугольника
```

```
ctx.stroke(); // отрисовка
```

```
ctx.fill(); // заливка
```

Задание js 9.2. Нарисовать вертикальный ряд окружностей радиусом 10 на расстоянии 100 от верхнего края экрана и с такими горизонтальными координатами 50, 80, 110, 140, ... , 290.

* раскрасить круги

Задание js 9.3. «Круги на воде».

Нарисуйте пару десятков concentric окружностей, то есть окружностей разного радиуса, но имеющих общий центр.

Задание js 9.4. Воспроизвести изображение при помощи программы:

